

第一模块：C语言与底层内存模型（嵌入式装备方向）

— 详细学习内容

（本模块内容已完成，此处保持不变，作为语言与内存基础。）

第二模块：MCU体系结构与中断机制（嵌入式装备方向）

— 超详细学习内容

本模块核心目标：建立“CPU如何真正执行代码”的底层认知，并理解中断如何改变控制流。
这一模块不是背API，而是理解执行模型、现场保护、优先级与实时性来源。

A. 基础知识（执行模型层）

A 1 . Cortex-M 处理器核心寄存器

必须熟悉的寄存器：

- PC（Program Counter）：下一条执行指令地址
- SP（Stack Pointer）：当前栈顶
- LR（Link Register）：函数返回地址
- xPSR：程序状态寄存器

理解重点：

CPU执行本质是：

取指 → 解码 → 执行 → 更新PC

中断发生时：

不是普通函数调用，而是 **硬件修改PC**。

A 2 . MSP 与 PSP（双栈模型）

Cortex-M 支持两个栈：

- MSP：主栈，默认用于中断
- PSP：进程栈，RTOS常使用

为什么要双栈：

- 防止任务栈破坏中断现场
- 提高系统稳定性

学习重点：

理解 CONTROL 寄存器如何切换栈。

A 3 . 启动流程 (Boot 到 main)

典型流程：

1) 读取向量表第 0 项 → 初始化 MSP 2) 读取向量表第 1 项 → 跳转 Reset_Handler 3) 复制 .data 段
4) 清零 .bss 段 5) 调用 main()

你必须会解释：

为什么全局变量默认是 0 。

A 4 . 向量表 (Vector Table)

本质：

一个函数地址数组。

结构：

[0] 初始栈地址
[1] Reset_Handler
[2] NMI_Handler
...

高级知识：

向量表可以重定位到 RAM。

用途：

- Bootloader
 - 动态中断重映射
-

B. 中断机制核心原理

B 1 . 中断触发全过程（必须能完整描述）

当中断到来：

1) CPU暂停当前执行流 2) 自动压栈：

- R 0 -R 3
- R 1 2
- LR
- PC
- xPSR

3) 读取向量表地址 4) 跳转 ISR 5) 执行 BX LR 返回 6) 自动弹栈恢复

重点理解：

这是 **硬件自动完成的上下文切换**。

B 2 . 为什么 ISR 不能写复杂逻辑

原因：

- ISR 会阻塞其他中断
- 影响实时性
- 增加栈消耗

正确模型：

ISR 只做：

- 读寄存器
- 放队列
- 设置标志

B 3 . NVIC 优先级模型（核心难点）

两个概念：

- 抢占优先级
- 子优先级（响应顺序）

关键理解：

数字越小优先级越高。

学习重点：

- 为什么优先级分组会影响中断嵌套
- BASEPRI 如何屏蔽低优先级中断

B 4 . 中断嵌套与尾链 (Tail Chaining)

Tail Chaining:

一个中断结束后直接进入下一个中断，减少栈操作。

意义：

- 提高响应效率
- 降低上下文切换开销

C. 进阶知识（理解实时性来源）

C 1 . HardFault 产生原因（必须掌握）

常见来源：

- 未对齐访问
- 空指针
- 栈溢出
- 执行非法地址

工程思路：

读取堆栈中保存的 PC 定位问题。

C 2 . 临界区与中断屏蔽

三种方式：

- PRIMASK：关闭所有可屏蔽中断
- BASEPRI：屏蔽低优先级中断
- FAULTMASK：极少使用

注意事项：

不要长时间关中断。

C 3 . 延迟中断 (PendSV)

用途：

- RTOS任务切换

特点：

- 设置最低优先级
 - 避免打断实时任务
-

C 4 . SysTick 定时器

作用：

- 系统节拍
- 时间基准

学习重点：

理解为什么 RTOS 依赖 SysTick。

D. 高阶知识（系统级理解）

D 1 . 中断延迟 (Latency) 从哪里来

来源：

- 当前指令执行时间
- 关中断时间
- 高优先级ISR占用

优化思路：

- ISR最小化
 - 使用DMA代替轮询
-

D 2 . 中断抖动 (Jitter)

原因:

- 优先级竞争
- Cache 或总线争用

工程影响:

- 控制系统稳定性
-

D 3 . 栈溢出分析

现象:

- 随机HardFault

原因:

- ISR过多
- 局部变量过大

建议:

- 启用栈填充检测
-

E. 高频技术考点 (标准答案)

E 1 . 中断发生时CPU做了什么?

标准答案:

CPU自动压栈保存现场, 然后根据向量表跳转ISR, 执行完后自动恢复现场返回。

E 2 . 为什么中断里不能延时?

标准答案:

会阻塞其他中断, 破坏实时性, 并可能导致系统响应失败。

E 3 . 优先级数字越大还是越小优先级高？

标准答案：

大多数 Cortex-M 中，数字越小优先级越高。

E 4 . HardFault 如何定位？

标准答案：

通过读取异常堆栈中的 PC 与 LR，还原出错指令地址。

F. 注意事项（真实工程经验）

- 1) 不要在 ISR 中调用 printf。
 - 2) 不要在 ISR 中 malloc。
 - 3) 优先级设计要有层级，而不是全部最高。
 - 4) 共享变量必须 volatile + 临界区。
 - 5) 栈大小必须评估最坏情况。
-

G. 深度学习建议（非常重要）

如果你想真正掌握这一模块，请完成以下练习：

- 手写一个最小向量表
 - 写一个GPIO中断并观察堆栈变化
 - 人工触发PendSV理解上下文切换
-

到这里，你已经从“会用中断”升级到“理解CPU如何执行中断”。